

Session / EP Module

Core Path & Core Design

Derived from docs/design/session/2_coreloop.md v2.4

What this module solves

Turn “*the user wants an EP and a device*” into “*an `ort.InferenceSession` bound to the right `OrtEpDevice` handle.*”

Two end-to-end paths:

- **Path A** — one user intent → one session
- **Path B** — enumerate all → render report

<i>CLI args</i> ↓	<i>typed intent</i> ↓	<i>resolved target</i> ↓	<i>registered EP</i> ↓	<i>matched device pair</i> ↓
<i>InferenceSession</i>				

User Scenarios — 9 actions, 2 paths

Path A (single intent → single session) — 8 scenarios:

- **A.1–A.4** by-name: `--ep openvino --device gpu / --ep openvino / --device npu / bare`
- **A.5–A.6** by-listing-pick: `--ep openvino@pypi[ --device npu]`
- **P.1** programmatic SDK / **P.2** persisted JSON config

Path B (enumerate all → render) — 2 scenarios:

- **B.1** `winml sys --list-ep`
- **B.2** `winml sys --doctor` (**PROPOSED**)

All scenarios route through the same six-class taxonomy and seven-API surface. The CLI shape determines which path; the rest is shared.

Six Classes, One Role Each

Class	Role	Prefix
EPDeviceTarget	User intent — ep, device, optional source	—
EPDeviceSpec	Catalog row — what could exist for an EP	WinML*
EPEntry	Filesystem-discovery record	—
WinMLDevice	Adapter over <code>ort.0rtEpDevice</code> — single concrete class	WinML*
WinMLEP	Successful per-source registration aggregate	WinML*
WinMLEPDevice	Flat (ep, device) pair — mirror of <code>ort.0rtEpDevice</code>	WinML*

Naming rule: WinML* = predefined or system-generated (cannot be crafted from CLI strings; requires system API). No prefix = user-craftable (built from strings / JSON config / tests).

Core APIs — 7 Primitives

API	Returns
<code>discover_all_eps()</code>	<code>list[EPEntry]</code> — flat filesystem scan
<code>EPSource.resolve()</code>	<code>Iterator[EPEntry]</code> — per-subclass
<code>resolve_device(target)</code>	<code>EPDeviceTarget</code> — pure deduction
<code>wrap_ort_device(handle)</code>	<code>WinMLDevice</code> — adapter factory
<code>WinMLEPRegistry.register_ep(entry)</code>	<code>WinMLEP</code> — atomic, idempotent
<code>WinMLEPRegistry.auto_device(target)</code>	<code>WinMLEPDevice</code> — compound, retry-shadowed
<code>WinMLSession(...)</code>	<code>ort.InferenceSession</code> — Path A user entry

`register_ep` is atomic per-source. `auto_device` composes it with candidate-filter, source-tag dispatch, and shadowed-source retry.

Two Decompositions – Orthogonal Axes

Path = user-facing (which scenario the user is in). **Tier** = internal (which kind of work the layer is doing).

	Tier 1 – Discovery	Tier 2 – Registration	Tier 3 – Validation
Cost / side-effect	filesystem scan only (no DLL load)	register_execution_provider_dlls (one DLL per call)	smoke-test inference (small model run)
Where it lives	discover_all_eps(WinMLEPRegistry.resolve(WinMLEPDevice))	register_execution_provider_dlls(WinMLEPRegistry.resolve(WinMLEPDevice))	EPDoctor.diagnose() (PROPOSED)
Path A uses?	✓ via _entries cache	✓ auto_device → one WinMLEPDevice	✗
Path B uses?	✓ full broad walk	✓ per-EPEntry inline loop	✓ (--doctor only)

Every path consumes Tier 1 + Tier 2; only Path B's - - doctor tail consumes Tier 3.

Path A — User Intent → Session

CLI args / JSON config / Python SDK



▼ EPDeviceTarget(ep, device, source?)

resolve_device(target)

pure deduction; fills "auto",
validates source against discovery



▼ EPDeviceTarget – concrete, validated

registry.auto_device(target)

filter EPEntries by ep+source,
retry-shadowed on failure



▼ WinMLEPDevice(ep: WinMLEP, device: WinMLDevice)

WinMLSession(onnx_path, ep_device)

eager InferenceSession build

|

▼ ort.InferenceSession

Path B — Enumerate All → Report

CLI: winml sys --list-ep



```
for entry in discover_all_eps():
    try:
        ep = registry.register_ep(
            entry)
        results.append(ep)
    except WinMLEPRegistrationFailed:
        failures.append((entry, e))
```

broad scan;
each entry triggers
one DLL load via
register_ep

failures captured
as data



▼ (list[WinMLEP], list[(EPEntry, Exception)])

render: primary /

per-row grouping with live

| shadowed / incompatible |

device facts from WinMLDevice

Scenario B — --ep <name>@<source-tag>

Seven canonical source tags (closed set):

bundled · pypi · nuget · msix-microsoft · msix-workload · winml-catalog · directory

Round-trip:

```
$ winml sys --list-ep
```

```
OpenVIN0ExecutionProvider
```

```
[primary]    PyPI      onnxruntime-ep-openvino 1.4.1
```

```
[shadowed]   Catalog (catalog default)
```

```
[shadowed]   MSIX      MicrosoftCorporationII.WinML.Intel.OpenVIN0.EP.1.8
```

```
$ winml perf --ep openvino@msix-microsoft --device npu # ← user pastes back
```

CLI parser splits on @; EPDeviceTarget.__post_init__ validates the tag against VALID_SOURCE_TAGS. auto_device filters discovered candidates by _entry_source_tag(e) == target.source.

Construction-Time Validation

EPDeviceTarget.__post_init__ rejects bad input at construction — CLI parse, JSON load, test fixture — not 5 layers deeper.

Field	Allowed
device	"auto" or one of VALID_DEVICES = {"npu", "gpu", "cpu"}
ep	"auto" or known_ep_short_names() (derived from catalog, not hardcoded)
source	None or one of VALID_SOURCE_TAGS (the 7 canonical tags)

Structural only — checks shape, not environment fit. Whether the EP is registered on this host, or whether the source tag has a match in discover_all_eps(), is resolve_device()'s job.

No hardcoded EP names (CLAUDE.md cardinal rule 1) — known_ep_short_names() derives from _SHORT_TO_FULL.

Key Design Decisions

- **Two paths, one taxonomy** — Path A (single) and Path B (broad) share the same six classes and seven APIs
- **Atomic + compound registry** — `register_ep(entry)` → WinMLEP is per-source atomic; `auto_device(target)` → WinMLEPDevice composes filter + retry-shadowed
- **Failures-as-data on Path B** — (EPEntry, Exception) pairs alongside successful WinMLEPs; renderer renders both
- **No back-compat shims** — hard-break naming, signature changes, JSON shape changes (forward-compat via `from_dict`'s `.get()` pattern)
- **Single discovery cache** — `WinMLEPRegistry.__init__` calls `discover_all_eps()` once; `auto_device` and `resolve_device` read from the cached `self._entries` instead of re-scanning

- **Single concrete WinMLDevice class** — not ABC + 5 subclasses (over- engineered for v1; dispatch via `self._ort.ep_name`)

What's next

Pending follow-ups (backlogged tasks):

- **Batch F** — CLI @ parser fully wired into `commands/perf.py` / `commands/compile.py` (currently constructs `EPDeviceTarget` without splitting @)
- **Batch G** — fix the `--list-ep duplicate-MSIX-DLL` render; wire `IncompatibleListingPick` / `AmbiguousListingPick` raises
- **Batch H** — `WinMLEPDevice.__post_init__` invariant assertion (`.device` in `.ep.devices`)
- **Future PR** — `--doctor smoke` test (Path B.2 PROPOSED in design only)

Out of scope: Tier 1/2/3 internals (`3_design_ep.md`), `WinMLDevice` dispatch tables (`4_winml_device.md`), monitor's per-op tracing (`monitor/2_coreloop.md`).

Q&A

Full spec: docs/design/session/2_coreloop.md v2.4

Canonical class reference: docs/design/session/3_design_classes.md v1.1